ReasonLink: Characterizing the interface between Neural–Symbolic Accelerators

Vayun Gupta (vg2684), Aaron Mano Cherian (amc2535)
COMS 6998: AI-Native Computing | Fall 2026
Project Proposal | Due October 5, 2026, 11:59 PM ET

Track / topic: A: Computing for AI; Computing Systems for Neuro-Symbolic and Compositional AI.

I. Problem and Motivation

Problem. Neuro-symbolic systems combine dense neural computation with irregular symbolic reasoning. Their accelerators consume different representations: neural outputs must be converted into discrete variables or constraints, transferred, and synchronized before symbolic execution. These handoff costs may become a significant fraction of runtime once both computational stages are accelerated.

Motivation. Faster individual kernels do not guarantee proportional end-to-end speedup. Format conversion, redundant memory transfers, and coordination can leave accelerators idle. Understanding these costs can guide interface designs that translate component-level acceleration into system-level gains.

Research question. After accelerating both ends of a neuro-symbolic pipeline, does the interface become the bottleneck? We will characterize representation conversion, synchronization, and data movement between Gemmini and SAT-Accel, then evaluate mechanisms that reduce their exposed cost. MNIST-based Sudoku will provide a lightweight initial workload; DataBack is an optional larger evaluation. Our goal is to identify when the handoff limits performance and improve it relative to serial and double-buffered FIFO baselines.

II. Related Work

CogSys [1] addresses heterogeneous compute and irregular memory access through algorithm–hardware co-design. Reconfigurable neural/symbolic processing elements, specialized dataflow, and workload-aware scheduling support both kernel types within a unified architecture, improving utilization and end-to-end performance.

REASON [2] targets probabilistic logical reasoning as a major source of inefficiency. It introduces a common graph representation, a reconfigurable processing architecture, and a multi-level pipeline integrated with GPU processing. These systems demonstrate the value of coordinated execution; our focus is the boundary between independently optimized accelerators after both stages have been accelerated.

We build on Gemmini [3], a parameterizable, systolic-array DNN accelerator integrated with RISC-V processors through Chipyard, and SAT-Accel [4], a Vitis HLS SAT-solving accelerator. Together, they enable a concrete study of conversion, communication, and synchronization in a heterogeneous SoC. Our contribution is an interface characterization and improved dataflow mechanism rather than another standalone compute accelerator.

III. Proposed Approach

We will build the pipeline in Fig. 1 using Gemmini in Chipyard and exported SAT-Accel RTL with a Chisel wrapper and adapted memory/control interfaces. In MNIST-based Sudoku, Gemmini recognizes images of given digits; the interface converts predictions into constraints for SAT-Accel. Cell positions and blank cells are supplied. Fixed Sudoku constraints can be cached, while per-puzzle givens form a small update payload.

The serial baseline completes inference before converting, transferring, and consuming its outputs. The double-buffered FIFO baseline alternates two buffers so production and transfer of one instance overlap symbolic execution of another. Both use explicit completion and buffer-ownership signals to preserve correctness under backpressure.

The proposed interface will explore compact intermediate representations, buffering, delta transfers, and producer–consumer overlap. Delta transfers will send changed givens relative to cached constraints instead of rebuilding the complete payload. We will investigate incremental conversion and solver-state loading as neural outputs arrive; final solving must respect the complete constraint set. The goal is to reduce bytes moved, unnecessary synchronization, and accelerator idle time.

If time permits, we will evaluate a capacity-filtered DataBack subset [5] using a NeuroBack-style variable-phase predictor [6]. This handoff supplies heuristic phase hints rather than hard constraints, preserving the original formula. Supported dense operations will run on Gemmini; any CPU preprocessing or unsupported operations will remain included in runtime. The expected contribution is a measured map of interface bottlenecks and a mechanism that improves end-to-end performance.

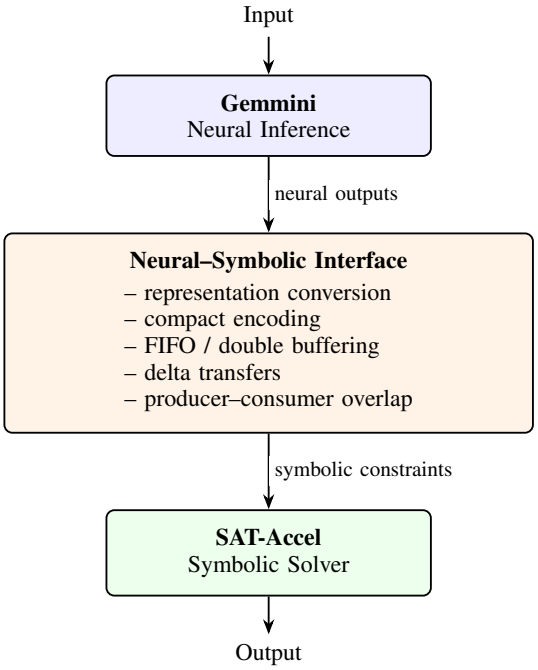


Fig. 1. Proposed neural–symbolic pipeline.

IV. Evaluation Plan

We will compare the proposed interface with serial execution and double-buffered FIFO communication on MNIST-based Sudoku, with DataBack as an optional scale-up. Comparisons will use identical neural predictions, constraints, and solver settings to isolate interface effects. Inputs will begin in host memory, and timing will end when the result returns to the host; runtime conversion, transfers, and coordination will be included.

Metrics will include end-to-end latency and throughput, representation-conversion time, bytes transferred, synchronization and stall cycles, and utilization of both accelerators. We will report the fraction of runtime attributable to the handoff. Serial measurements will use disjoint intervals; overlapped runs will distinguish total activity from exposed critical-path delays to avoid double counting. Stall traces and service rates will establish whether the interface actually limits throughput.

We will vary workload size, FIFO capacity, and transfer granularity, and evaluate compact encoding, buffering, delta transfers, and overlap individually and together. Ablations will identify which mechanisms account for speedup and when longer symbolic execution amortizes communication. Sudoku accuracy and SAT assignments will be checked against reference outputs; interface configurations must preserve results. Instances exceeding hardware capacity and solver timeouts will be reported separately.

V. Timeline and Team Responsibilities

Date	Planned work and deliverables
Oct 19	Set up Gemmini/Chipyard and SAT-Accel; define the intermediate representation; establish the Sudoku pipeline, serial/FIFO baselines, and instrumentation.
Nov 2	Characterize baseline costs; prototype compact encoding, buffering, and overlap; prepare preliminary results for the Nov 6 presentation.
Nov 23	Complete the interface; compare individual optimizations across workload sizes and configurations; begin DataBack if feasible.
Dec 7	Finish experiments and ablations; identify bottleneck regimes; prepare figures and conclusions for the Dec 11 poster.
Dec 18	Incorporate poster feedback; finalize implementation, analysis, documentation, report, and reproducible project artifacts.

Team roles. Both team members will contribute across Gemmini/Chipyard integration, SAT-Accel, the interface, and evaluation. Implementation and experimental tasks will be split for parallel work, while design decisions, debugging, experiments, and analysis remain shared. Regular review of each other's work will ensure both maintain a complete understanding of the pipeline.

REFERENCES

[1] Z. Wan, H. Yang, R. Raj, C.-K. Liu, A. Samajdar, A. Raychowdhury, and T. Krishna, "CogSys: Efficient and scalable neurosymbolic cognition system via algorithm-hardware co-design," in *2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025. [Online]. Available: https://zishenwan.github.io/publication/HPCA25_CogSys.pdf

[2] Z. Wan, C.-K. Liu, J. Qian, H. Yang, A. Raychowdhury, and T. Krishna, "REASON: Accelerating probabilistic logical reasoning for scalable neuro-symbolic intelligence," in *2026 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026. [Online]. Available: https://arxiv.org/abs/2601.20784

[3] UC Berkeley Architecture Research, "Gemmini: Full-system, full-stack DNN hardware exploration," Project documentation, accessed October 4, 2026. [Online]. Available: https://github.com/ucb-bar/gemmini/blob/master/README.md

[4] M. Lo, M.-C. F. Chang, and J. Cong, "SAT-Accel: A modern SAT solver on a FPGA," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 234–246. [Online]. Available: https://github.com/milo168/FPGA25_SAT_Accel

[5] W. Wang and Y. Hu, "DataBack: Dataset of SAT formulas and backbone variable phases," Hugging Face dataset, accessed October 4, 2026. [Online]. Available: https://huggingface.co/datasets/neuroback/DataBack

[6] W. Wang, Y. Hu, M. Tiwari, S. Khurshid, K. McMillan, and R. Miikkulainen, "NeuroBack: Improving CDCL SAT solving using graph neural networks," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/2f27964513a28d034530bfdd117ea31d-Abstract-Conference.html